

RAPPORT DU TRAVAIL RÉALISÉ

Projet 3 : Gestion des Candidatures

Application RIF-Stages

Stack Technologique Validée :

React · Node.js (Express) · MySQL (XAMPP)

Réalisé par :
ERRACHI Moedem

Table des matières

1	Maquettage et User Journeys (Partie 1)	2
1.1	Gestion des User Journeys (Parcours Utilisateurs)	2
1.2	Maquettage Fonctionnel des Interfaces	2
2	Implémentation de la Base de Données (Partie 2)	3
2.1	Modélisation Conceptuelle : Diagramme de Classes	3
2.2	Structure Physique relationnelle (MySQL)	4
2.3	Script SQL de Déploiement	4
2.3.1	Preuve d'Intégration sur phpMyAdmin	6
3	Développement de l'API Backend (Partie 3)	6
3.1	Architecture des Routes et Logique Métier	6
3.2	Contrôleur Express d'Évolution de Statut	6
3.3	Protocole de Vérification de l'Endpoint	6
3.3.1	Validation des Endpoints API	7
4	Automatisation et Notifications (Partie 4)	7
4.1	Intégration du SDK Mailtrap et Variables Globales	7
4.2	Code du Service de Messagerie	7

Partie 1: Maquettage et User Journeys (Partie 1)

1.1 Gestion des User Journeys (Parcours Utilisateurs)

Pour concevoir au mieux les fonctionnalités du **Projet 3 : Gestion des Candidatures**, l'expérience utilisateur a été cartographiée selon deux parcours fondamentaux :

- **Le Parcours Étudiant (Candidat)** : L'utilisateur authentifié consulte les offres de stage, accède au formulaire de postulation, téléverse son CV en format PDF, rédige une lettre de motivation optionnelle, puis valide son envoi. Le système bloque automatiquement toute tentative de double postulation.
- **Le Parcours Recruteur (RH)** : Le responsable RH accède à la liste consolidée des candidatures liées à ses offres. Il peut examiner les pièces jointes, attribuer une évaluation textuelle et faire basculer l'état du dossier (*En revue*, *Acceptée*, *Refusée*), ce qui déclenche instantanément le traitement de notification.

1.2 Maquettage Fonctionnel des Interfaces

Les maquettes fonctionnelles (wireframes) des écrans cibles ont été modélisées afin de structurer l'agencement visuel de l'application. L'accent a été mis sur la clarté du tableau de bord RH pour simplifier la prise de décision lors du changement de statut d'un candidat.

L'intégralité des interfaces graphiques ainsi que le prototype interactif simulant la navigation des utilisateurs sont consultables en ligne à l'adresse suivante :

Lien du Prototype : [RIF Stages App – Figma](#)

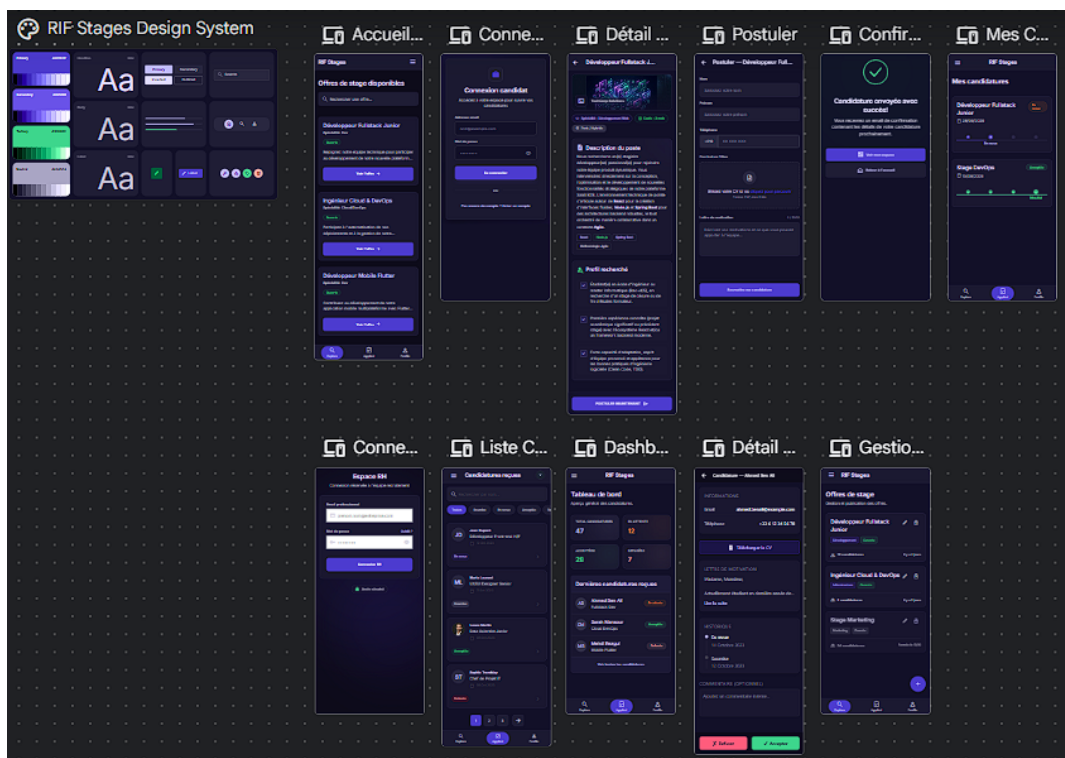


FIGURE 1.1 – Maquettes fonctionnelles et cinématique des parcours de la plateforme

Partie 2: Implémentation de la Base de Données (Partie 2)

2.1 Modélisation Conceptuelle : Diagramme de Classes

Avant de transcrire la structure en tables SQL, un diagramme de classes UML a été réalisé. Il formalise les entités du système (**User**, **Offre**, **Candidature**), leurs attributs respectifs ainsi que les multiplicités des associations (un utilisateur de type RH peut créer plusieurs offres, une candidature lie exactement un candidat à une offre spécifique).

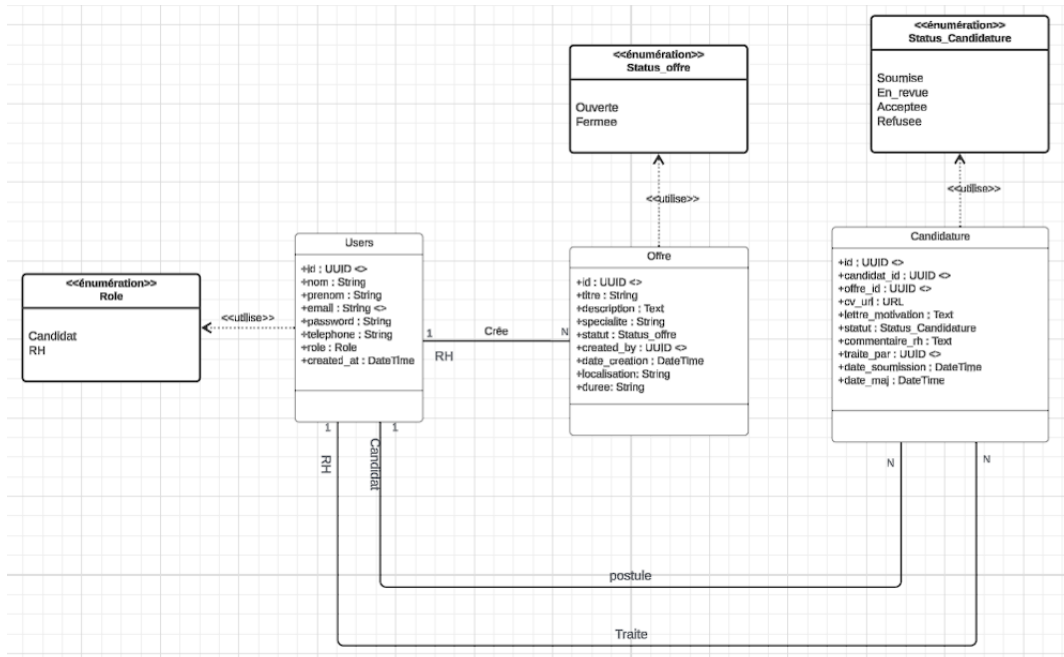


FIGURE 2.1 – Diagramme de classes UML du module de gestion des candidatures

2.2 Structure Physique relationnelle (MySQL)

L'ensemble de la couche de persistance a été configuré sur un serveur local **MySQL** via l'environnement **XAMPP**. Une contrainte unique composite `UNIQUE(candidat_id, offre_id)` a été expressément injectée au sein de la table des candidatures afin d'interdire mathématiquement à un candidat de postuler plusieurs fois à la même offre.

2.3 Script SQL de Déploiement

Le code suivant détaille la création des schémas de données respectant l'intégrité référentielle par clés étrangères :

```

1 CREATE TABLE User (
2     id INT AUTO_INCREMENT PRIMARY KEY,
3     nom VARCHAR(100) NOT NULL,
4     prenom VARCHAR(100) NOT NULL,
5     email VARCHAR(150) UNIQUE NOT NULL,
6     password_hash VARCHAR(255) NOT NULL,
7     role ENUM('candidat', 'rh', 'admin') NOT NULL DEFAULT 'candidat',
8     created_at DATETIME DEFAULT CURRENT_TIMESTAMP
9 );
10
11 CREATE TABLE Offres (
12     id INT AUTO_INCREMENT PRIMARY KEY,
13     titre VARCHAR(150) NOT NULL,
14     description TEXT NOT NULL,

```

```
15     specialite VARCHAR(100) NOT NULL,
16     statut ENUM('ouverte', 'fermee') DEFAULT 'ouverte',
17     created_by INT NOT NULL,
18     date_creation DATETIME DEFAULT CURRENT_TIMESTAMP,
19     FOREIGN KEY (created_by) REFERENCES User(id) ON DELETE CASCADE
20 );
21
22 CREATE TABLE Candidature (
23     id INT AUTO_INCREMENT PRIMARY KEY,
24     candidat_id INT NOT NULL,
25     offre_id INT NOT NULL,
26     cv_url VARCHAR(255) NOT NULL,
27     lettre_motivation TEXT,
28     statut ENUM('soumise', 'en_revue', 'acceptee', 'refusee') DEFAULT
29     'soumise',
30     commentaire_rh TEXT NULL,
31     traite_par INT NULL,
32     date_soumission DATETIME DEFAULT CURRENT_TIMESTAMP,
33
34     UNIQUE(candidat_id, offre_id),
35     FOREIGN KEY (candidat_id) REFERENCES User(id) ON DELETE CASCADE,
36     FOREIGN KEY (offre_id) REFERENCES Offre(id) ON DELETE CASCADE,
37     FOREIGN KEY (traite_par) REFERENCES User(id) ON DELETE SET NULL
38 );
```

Listing 2.1 – Script de création des tables relationnelles

2.3.1 Preuve d'Intégration sur phpMyAdmin

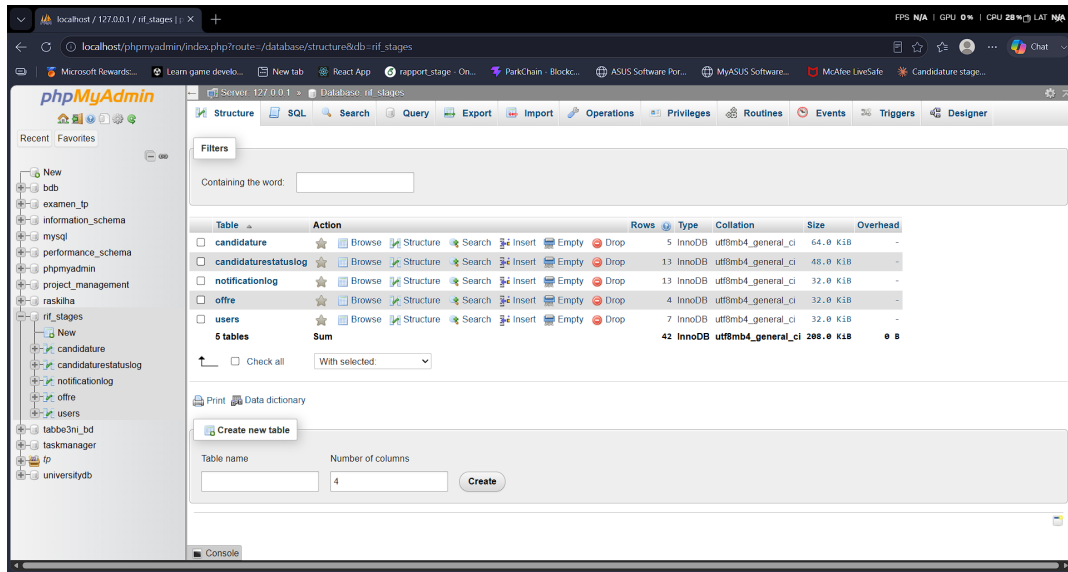


FIGURE 2.2 – Visualisation de l'architecture des données sous XAMPP

Partie 3: Développement de l'API Backend (Partie 3)

3.1 Architecture des Routes et Logique Métier

Le serveur développé en **Node.js (Express)** expose les points d'accès (endpoints) nécessaires au traitement des flux. Lorsqu'un recruteur modifie le statut d'une candidature, l'API valide la requête, met à jour la ligne SQL correspondante, puis extrait dynamiquement l'adresse courriel mockée du candidat pour alimenter l'étape suivante.

3.2 Contrôleur Express d'Évolution de Statut

L'implémentation de la logique métier repose sur un contrôleur Express dédié. Ce dernier encapsule la double responsabilité de valider les transitions d'état en base de données (passages aux statuts *en revue*, *acceptée* ou *refusée*) et d'orchestrer immédiatement l'appel asynchrone vers le service de notification externe.

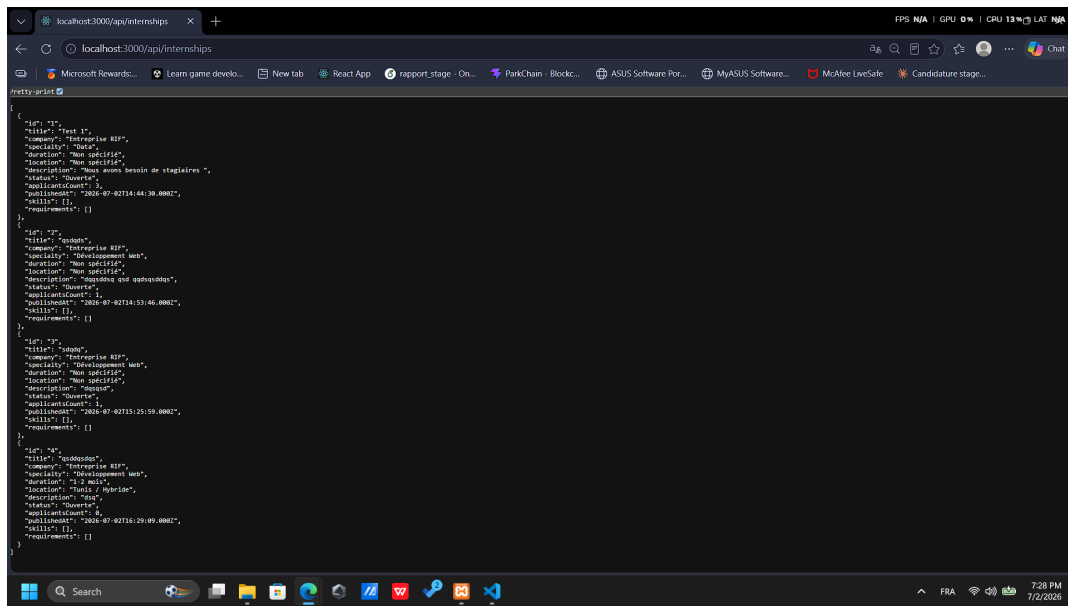
3.3 Protocole de Vérification de l'Endpoint

Pour valider le bon fonctionnement de notre API, un test d'intégration a été réalisé à l'aide d'un client HTTP (Postman / Insomnia) selon la procédure suivante :

- **Type de Requête** : PUT (ou PATCH)
- **URL ciblée** : `http://localhost:3000/api/candidatures/1/statut`
- **Corps de la requête (JSON Object)** :

- "statut": "acceptee"
- "commentaire_rh": "Excellent profil technique, entretien à planifier."
- **Comportement attendu** : Le serveur doit renvoyer un code statut 200 OK accompagné d'un message de succès au format JSON, confirmer la modification en base de données et déclencher l'envoi du mail via Mailtrap.

3.3.1 Validation des Endpoints API



```

{
  "I1": {
    "title": "Test I1",
    "company": "Entreprise RHP",
    "specialty": "Test",
    "duration": "Non spécifiée",
    "location": "Non spécifiée",
    "description": "Non avons besoin de stagiaires",
    "status": "Ouverte",
    "applicationcount": 0,
    "publishedat": "2025-07-02T14:44:30.000Z",
    "skills": [],
    "requirements": []
  },
  "I2": {
    "title": "test2",
    "company": "Entreprise RHP",
    "specialty": "Développement Web",
    "duration": "Non spécifiée",
    "location": "Non spécifiée",
    "description": "Apprendre aux qdusdus",
    "status": "Ouverte",
    "applicationcount": 1,
    "publishedat": "2025-07-02T14:53:46.000Z",
    "skills": [],
    "requirements": []
  },
  "I3": {
    "title": "test3",
    "company": "Entreprise RHP",
    "specialty": "Développement Web",
    "duration": "Non spécifiée",
    "location": "Non spécifiée",
    "description": "Apprend",
    "status": "Ouverte",
    "applicationcount": 1,
    "publishedat": "2025-07-02T15:25:59.000Z",
    "skills": [],
    "requirements": []
  },
  "I4": {
    "title": "test4",
    "company": "Entreprise RHP",
    "specialty": "Développement Web",
    "duration": "1.2 mois",
    "location": "Tunis / Morice",
    "description": "App",
    "status": "Ouverte",
    "applicationcount": 0,
    "publishedat": "2025-07-02T16:29:09.000Z",
    "skills": [],
    "requirements": []
  }
}

```

FIGURE 3.1 – Traces d'exécution et logs réseau du serveur Node.js lors de la vérification de l'endpoint

Partie 4: Automatisation et Notifications (Partie 4)

4.1 Intégration du SDK Mailtrap et Variables Globales

Pour valider l'acheminement des flux à l'aide de données de tests et d'e-mails factices (mocks), le SDK officiel **Mailtrap** a été couplé à **Nodemailer**. L'isolation stricte des identifiants (Token API) est assurée via l'utilisation d'un fichier de variables d'environnement (.env).

4.2 Code du Service de Messagerie

Le module backend gère dynamiquement la génération et l'injection de modèles HTML responsives. Le contenu s'adapte en temps réel selon l'évaluation fournie par le recruteur, garantissant une communication fluide et transparente avec le candidat.